

F17_v2 — a primary-skill regime HMM for the meta-model

What this is. A label-aware Hidden Markov Model that reads each primary signal's recent **environment and track record**, infers a latent **skill regime** (GOOD / NEUTRAL / BAD), and hands the meta-model a compact, persistent "is the primary working right now?" summary. Implemented as the model-layer family **f17_v2** (`src/stml/model/hmm_features_v2.py`), the successor to the feature-layer F17 (`src/stml/metamodel/regime_features_hmm.py`).

Pointers. Code `src/stml/model/hmm_features_v2.py` · tests `tests/model/test_hmm_features_v2.py` · wired into `notebooks/jay/metamodel.ipynb` · course refs: Lecture 3 (HMM forward / Baum-Welch), Session 3 (HMMs), Session 5 (metamodel signal-filtering), Session 2 (cluster-level importance).

0. TL;DR

The meta-labeling question — "is this trade worth taking?" — is, structurally, "are we in a regime where the primary has edge, and will this trade resolve profitably?" "Has edge" is a **latent, persistent** state, which is exactly what a first-order Markov chain models and a plain mixture (GMM) cannot. We fit a 3-state Gaussian HMM **per instrument** over an 11-dimensional observation vector that blends market environment, signal character, and — the novel part vs F17 — the primary's recent realised performance (rolling hit-rate / expected-return / Sharpe). The filtered (causal) regime posteriors and one-step-ahead predictives become `f17_v2_*` features the meta-model uses to **condition its trust on the latent regime**. Everything is frozen on FE-train and proven causal by two tests (truncation-invariance of the posterior; resolved-before- entry of the performance channel).

1. Setting — the meta-model as a latent-regime problem

Premise. A meta-model filters a primary signal: it learns to drop signals fired in regimes where the primary is unreliable and keep those where it has edge (Session 5). Two facts hide in that sentence. First, "reliable / unreliable" is **latent** — never observed, only inferred. Second, regimes **persist**: if the primary worked last week it probably still works today. Persistence of a hidden discrete state is a **first-order Markov chain** — precisely the structure a GMM lacks (it treats each point as i.i.d.) and an HMM supplies through its transition matrix.

Our hidden state. $H_i \in \{\text{GOOD}, \text{NEUTRAL}, \text{BAD}\}$ ($M = 3$) is the primary's **skill regime** at signal i :

Regime	Meaning	Identified by
GOOD	primary has genuine edge	FE-train hit-rate > 0.5 (highest)
NEUTRAL	no clear edge	hit-rate $\approx$ 0.5
BAD	primary actively unreliable	hit-rate < 0.5 (lowest)

The states emerge unsupervised from the observation clustering; we name them post-fit by ordering on the FE-train per-state hit-rate (`HmmV2Bundle.state_hit_rate`, GOOD = column 0). The ordering is **frozen on train** (it depends on labels, so re-deriving it on val/test would leak) — the same discipline as F17's vol-ascending order.

Ingredients, in our setting.

- **Transition matrix** Q — carries the persistence. The diagonal is large (sticky regimes); this is precisely why the HMM beats a memoryless tabular classifier — a recent run of wins is evidence we are **still** in GOOD, information a per-row model cannot represent.
- **Initial distribution** π and **multivariate Gaussian emissions** $N(\mu_m, \Sigma_m)$ over the observation vector (§2).
- Fit by **Baum-Welch** (EM) on FE-train events; the **forward algorithm** gives the **filtered** posterior $\xi(i, \cdot) = P(H_i | \text{obs}_{\{0..i\}})$ — strictly causal, never the smoothed forward-backward posterior (which conditions on the future and would leak).
- Fit **per instrument** (the 11 futures are near-independent, cross-asset corr $\approx$ 0.09); thin names are **pooled** (§4).

The three classical HMM problems → our tasks. Likelihood (forward) → choosing M (§6); learning (Baum-Welch) → fitting Q, π, μ, Σ ; filtering + one-step Q → producing the regime features the meta-model consumes.

Relation to F17. F17 already fits a causal Gaussian HMM, but only on market (`ret, vol`) — a pure market-regime detector. `f17_v2`'s novelty is the **performance channel** (RHR/RER/RS): the HMM now sees how the primary itself has been doing, turning a market-regime detector into a skill-regime detector. That channel needs triple-barrier labels, which is why `f17_v2` lives in the model layer, not the model-free feature matrix.

2. The observation vector — what it is and what it means

For each signal i on instrument k , the 11-dimensional observation o_i answers "what environment is this signal firing into, and how has the primary been doing?" in three blocks.

Block 1 — market environment (what kind of tape):

dim	meaning	our column
vol	realised volatility (calm ↔ turbulent)	<code>f2_vol_20</code> ÷ $\sqrt{252}$ (de-annualised)
trend	trending vs choppy	<code>f12_hurst_100</code>
autocorr	lag-1 return autocorrelation (momentum ↔ mean-reversion)	<code>f12_autocorr_21</code>
ret_1/5/20	short/medium momentum at entry	recomputed log-returns from <code>close_wide</code>

Because our primary is short-horizon **mean-reversion / counter-trend**, negative autocorr and the vol level are especially diagnostic of when it should work.

Block 2 — signal character (how the primary is behaving):

dim	meaning	our column
conf	signal conviction (no primary probability exists; proxy by persistence)	f5_trailing_run_length
freq	how often the primary is firing (selective ↔ overtrading)	f5_participation_20

There is **no primary predicted-probability** — the signal is sign-only $s \in \{-1, 0, +1\}$ — so conf is a documented proxy; high firing freq often flags an indiscriminate (bad) regime.

Block 3 — primary performance (the direct skill read; the novel channel):

dim	meaning	source
rhr	rolling hit-rate of recently resolved trades	mean of bin over resolved priors
rer	rolling expected return	mean of ret
rs	rolling Sharpe of the primary	mean/std·√n of ret

RHR/RER/RS are computed over the **most-recently-resolved** window (default 20) of prior events — those whose triple barrier has **already closed by entry** ($t1[j] \leq \text{date}[i]$). This is the most direct evidence of GOOD vs BAD: a string of recent wins with positive expectancy is the GOOD regime. RER/RS add magnitude that RHR alone misses (a 0.6 hit-rate with tiny wins and fat losses is a different regime from 0.6 with fat wins).

Why this vector yields the regimes. The HMM clusters signals into states where these 11 features **co-move** characteristically — e.g. BAD $\approx$ high vol $\wedge$ low RHR $\wedge$ high freq, GOOD $\approx$ moderate vol $\wedge$ high RHR $\wedge$ selective. A **full** covariance Σ_m would capture those within- regime correlations jointly; we default to **diagonal** Σ_m for estimability (§4) and let the hit-rate ordering name the states. All dimensions are **standardised with FE-train-frozen** mean/ std before fitting (`HmmV2Bundle.feats_mean/feats_std`).

3. How it helps the meta-model — and which features we use

The HMM gives each signal a **compact, persistent summary** of "is the primary working now" that a tabular meta-model cannot easily reconstruct from raw features (it has no built-in Markov memory). Concretely, f17_v2 emits (column names exactly as produced):

Regime channel (`regime_columns()`): - f17_v2_regime_good / _neutral / _bad — filtered posteriors $P(\text{skill regime} \mid \text{history} \leq i)$. **The core feature:** high $P(\text{GOOD}) \rightarrow$ the meta-model should up-weight; high $P(\text{BAD}) \rightarrow$ drop. - f17_v2_regime_pred_good / _neutral / _bad — one-step-ahead predictive $\xi @ Q$ (the persistence- projected belief for the next signal). - f17_v2_regime_argmax — the most-likely regime (**nominal** — used as a diagnostic / one-hot, never fed as an ordinal column). - f17_v2_regime_entropy — posterior uncertainty (ambiguous/transitional regimes).

Performance channel (`perf_columns(window)`): - f17_v2_rhr_20 / _rer_20 / _rs_20 — informative standalone and the HMM's own inputs.

Mechanism. These columns merge into the meta-model design matrix (auto-selected by `dataset.select_features`'s $\wedge f \setminus d$ rule) alongside the 175 base features; the meta-model learns $P(y=1)$ from the union, **conditioning its trust on the latent regime** — realising

the meta- labeling premise. The GOOD posterior connects directly to the economic gate: take the trade only when expected value is positive, $p^* = L/(G+L)$ (Session 5 "adding zeros"), and the blind-primary confusion / NAV comparison is where the regime filter has to earn its keep.

Sanity expectations (checked in the notebook diagnostics): - counter-trend `f1_*` features and the GOOD posterior should rank high in cluster-importance; injected noise features should sink; - the decoded regimes must satisfy GOOD > NEUTRAL > BAD in realised hit-rate, with a **sticky** transition matrix (diagonal > 0.7) and multi-signal dwell times — else `M` is too large or the features are uninformative.

Honesty guards. Regime posteriors are an aid, not a guaranteed beat — and grading here is on **methodology, not performance**. Raw HMM posteriors are typically **miscalibrated** → calibrate on out-of-fold pairs (Platt; Session 5 Part 4) before using a probability as a size. And an "oracle" read of regime quality (filtering realised returns by realised labels) is an upper bound, not a backtest — see CLAUDE.md §11.

4. Design decisions & feasibility (adaptations to the literal spec)

Each default below is overridable; each adaptation has a rationale grounded in the data.

1. **Model-layer placement.** `f17_v2` lives in `src/stml/model/` (not the FE matrix) because the performance channel is label-dependent; this keeps `feature_matrix.parquet`, the catalog, and the 257 FE tests untouched.
2. **Diagonal covariance (default).** A full 11×11, 3-state Gaussian HMM has **≈239 free parameters** — infeasible against ~13%-effective-N data. Diagonal cuts it to **≈74**; `tied` and FE-train PCA are knobs to recover joint structure if desired.
3. **Pooled across sides.** The spec splits per (instrument, side) → 22 HMMs; we do **not** by default — `ret/bin` are already side-adjusted, the split halves already-thin data, and `ng1s` is **short-only** (its long HMM is empty). Per-side is an option for STRONG names.
4. **Scope: per-instrument for STRONG, pooled fallback for THIN.** A `__pooled__` bundle is always fit; thin/failed instruments route to it (`fit_transform_f17_v2(..., thin=("ho1s", "ng1s"))`).

tier	instruments	raw N	scope
STRONG (≥500)	es1s fesx1s hg1s nq1s pl1s rb1s si1s	556-636	per-instrument
ADEQUATE	cl1s (421), gc1s (160)	160-421	per-instrument (+ pooled fallback)
THIN	ho1s (63), ng1s (124, short-only)	<130	pooled

1. **M = 3 (default), tunable.** GOOD/NEUTRAL/BAD is interpretable and parsimonious for thin data; `n_states` is a knob and `select_n_states()` reports LL + BIC per `M` (§6).
2. **Warm-up = structural NaN.** Early events with `< min_count` resolved priors (and any `ok=False` scope) get all-NaN `f17_v2` columns — never filled; the meta-model's Preprocessor median-imputes per fold, exactly as for every other feature.

5. Leakage discipline (and the tests that prove it)

- **Frozen on FE-train** (`fe_train_end="2021-07-01"`): the HMM params, standardisation, and the GOOD/NEUTRAL/BAD ordering are estimated on `date ≤ FE-train` only, then applied causally — so CPCV consumes `f17_v2_*` as a frozen exogenous feature, exactly how F17 is treated. (A per-fold refit is the documented rigorous upgrade.)
 - **Filtered, never smoothed.** The served posterior is the forward-only `_causal_filtered_probs`; hmmlearn's smoothed `predict_proba` is used only in-sample for the hit-rate ordering.
 - **Resolved-before-entry.** RHR/RER/RS use only events with `t1[j] ≤ date[i]`; since `t1[j] > date[j]` always, an event never sees its own outcome.
 - **Two decisive tests** (`tests/model/test_hmm_features_v2.py`):
`test_filtered_posteriors_are_causal` (full vs future-truncated transform identical on the overlap, $\leq 1e-9$) and `test_rhr_resolved_before_entry` (flipping an unresolved/self/future outcome leaves the feature unchanged; flipping a resolved prior changes it).
-

6. Can the regime count M be a hyperparameter?

Yes. Two framings, both legitimate:

- **Separate (unsupervised-led)** — the framing that fits this design, since the HMM is a feature extractor: score each $M \in \{2,3,4,5\}$ by held-out forward log-likelihood + **BIC** ($= -2 \cdot \log L + k(M) \cdot \ln N_{\text{eff}}$), where $k(M)$ grows $\sim$ quadratically in the observation dimension d (`select_n_states()` reports this). Validate the BIC choice against downstream meta-model OOF AUC (mirroring how `barrier_search.py` scores label quality by downstream AUC). The penalty math is why thin names cannot support large M , and why $M=3$ is the data-honest default.
- **Joint** — if the HMM is instead used as a coupled generative meta-model (`predict_proba =  $\sum_m b_m \cdot \xi$`), M becomes one of its hyperparameters and is tuned by the existing two-stage CPCV (AUC $\rightarrow$ Sharpe) with no new machinery. That coupled model is a natural follow-on family for the 30-mark comparison.

We default to $M=3$ for the GOOD/NEUTRAL/BAD economics and **validate** rather than blindly tune.

7. Caveats & limitations

- **Oracle $\neq$ backtest.** Regime "quality" read off realised labels is an upper bound; pair any claim with the hold-out and the blind-primary comparison.
 - **Cross-instrument regime identity is interpretive.** A per-instrument GOOD and a pooled-fallback GOOD are both "highest hit-rate within their own fit," not a shared latent — don't over-read cross-instrument posterior comparisons (the instrument dummies let the pooled model condition).
 - **Thin names** (`ho1s` , `ng1s`) lean on the pooled fallback; report them per-instrument, never bury them (CLAUDE.md §7).
 - **Warm-up loss.** The first `min_count` events per instrument have NaN performance features and hence NaN regime posteriors — a structural, honest gap.
-

8. Wiring & artifacts

In `notebooks/jay/metamodel.ipynb`, after labels are built (`label_dev_per_instrument`) and before `make_xy` :

```
f17v2, bundles = fit_transform_f17_v2(dev_lab, lab_full, close_wide, n_states=3, window=20)
dev_lab = pd.concat([dev_lab.reset_index(drop=True), f17v2], axis=1)
# make_xy(dev_lab, select_features(dev_lab), ...) now auto-includes the f17_v2_* columns
```

The test partition reuses the frozen `bundles` via `transform_f17_v2(bundles, test_rows, labels_all, close_wide)`. Diagnostics (per-regime hit-rate ordering, transition stickiness, Viterbi dwell — diagnostic only, Viterbi is smoothing) and an explicit `f17_v2` bucket in the `cluster-importance` call complete the section.

References

López de Prado, *Advances in Financial Machine Learning* (meta-labeling Ch. 3, uniqueness Ch. 4). Course Lecture 3 (HMM evaluation / Baum–Welch / forward prediction), Session 3 (discrete HMMs), Session 5 (metamodel signal-filtering → ROC/AUC), Session 2 (cluster-level importance). See also this repo's `reports/jay/triple-barrier-label.md` (§11 labeling geometry) and `CLAUDE.md` §6–§7.